```
show_regexp('are you sure?', /e\?/) # => are you sur->e?<-
```

Now let's see what some of these characters mean if you use them without escaping them.

Anchors

By default, a regular expression will try to find the first match for the pattern in a string. Match /iss/ against the string "Mississippi," and it will find the substring "iss" starting at position 1 (the second character in the string). But what if you want to force a pattern to match only at the start or end of a string?

The patterns ^ and $ match the beginning and end of a line, respectively. These are often used to *anchor* a pattern match; for example, /^option/ matches the word *option* only if it appears at the start of a line. Similarly, the sequence \A matches the beginning of a string, and only at the beginning of a string, whereas ^ would match the first character after a newline even if it is not the beginning of the string. Similarly, you have \z and \Z which match the end of the entire string rather than the end of a line. The difference is that \Z matches the end of a string *unless* the string ends with \n, in which case it matches just before the \n.

```
str = "this is\nthe time"
show_regexp(str, /^the/)   # => this is\n->the<- time
show_regexp(str, /is$/)    # => this ->is<-\nthe time
show_regexp(str, /\Athis/) # => ->this<- is\nthe time
show_regexp(str, /\Athe/)  # => no match
```

The pattern \b is an anchor that matches a *word boundary*. A word boundary is the separation between a word character—an ASCII letter, a number, or an underscore—and something that isn't a word character. In the string "six o'clock", there are six word boundaries:

- Before the s—the beginning of the string is considered a non-word character.
- After the x—between the ASCII letter x and the space.
- Before the o—between the space and the letter o
- Before and after the ', because apostrophes are not word characters
- After the k—the end of the string is also considered a non word character.

You can see where word boundaries occur by replacing every instance of a word boundary with an * using gsub:

```
"six o'clock".gsub(/\b/, "*") # => "*six* *o*'*clock*"
```

The \B pattern is the inverse, it matches the boundary between any two characters that is not a word boundary. The string "six o'clock" also has six of those.

```
"six o'clock".gsub(/\B/, "*") # => "s*i*x o'c*l*o*c*k"
```

You use these anchors to limit a match to the beginning or end of a word (\b), or prevent a match from happening at the beginning or end of a word (\B).

```
show_regexp("this is\nthe time", /\bis/) # => this ->is<-\nthe time
show_regexp("this is\nthe time", /\Bis/) # => th->is<- is\nthe time
```

Character Classes

A *character class* is a set of characters between brackets: [*characters*]. The character class pattern matches any individual character between the brackets, with no delimiter separating them,